Fall EECS 348 Assignment 5

Happy Halloween, Mr. Zhe. I hope you're doing good.

Last time, we did our most complex analysis yet into a brute force depth first Sudoku algorithm. I was very impressed with Gemini Flash's output, but I suspected that the output was from the wealth of game-related examples Alphabet Inc has, because of its great amount of focus in games such as Chess and Go (the game, not the language, but they created that too).

Today's assignment will be a lot more light. We will be modifying an existing source code, of one Andy Garcia, to essentially just write the output of every solution to a file solutionN.txt. However, because I am inherenting code, I will do a pre-analysis before the LLM changes, and then do a post-analysis change of the LLMs. This is to cover all basis, and to discuss the LLM changes it makes sense to have a baseline. Especially because the changes occuring are so tiny, it makes little sense to just do analysis on a couple of lines of C++ code just writing cout to a file for each iteration.

Model/Access Point

Mr. Garcia says in his other sources

```
Other Sources - Base code generated by Google Gemini via the
        website. Claude used to assist with design changes by
        presenting ideas all code adjustment written by Andy Garcia.
        All comments rewritten by Andy Garcia
```

I assume, by my previous access, though unspecified, he means some version of Gemini Flash. As of October 24th, 2026, logging into Google Gemini through the web portal will give you 3.5 Flash Lite, and if logged in 3.6 Flash. I assume he used one of these two models.

Here, for completeness in the analysis, is the compilied file

```
/*
EECS 348 Assignment 4
Sudoku Puzzle Solver
Description - Solves the sudoku puzzle using a recursive depth-first
        search
with backtracking. Finds all solutions for each puzzle or reports if
        none exist

Inputs - The five given txt puzzle files. Each have a 9x9 grid with
        (_) being a
blank cell

Outputs - Each puzzle outputs the file name, the original puzzle,
        and either every solution found or "no solution found"

Collaborators - n/a
Other Sources - Base code generated by Google Gemini via the
        website. Claude used to assist
with design changes by presenting ideas all code adjustments written
        by Andy Garcia. All comments rewritten by Andy Garcia
```

```
Author - Andy Garcia

Creation Date - 10/14/26
Revision Date -
*/

#include <iostream> // Include input-output stream library for
        console printing
#include <fstream>  // Include file stream library for reading
        puzzle text files
#include <vector>   // Include vector library to store the 9x9 grid
        and list of solutions
#include <string>   // Include string library for handling file
        names and strings

class SudokuSolver
{                                       // Creates a class to make
        a SudokuSolver
private:                                // Make it private instead
        of public
    std::vector<std::vector<int>> board; // Use a 2d vector to
        represent the grid of size 9x9

    bool isValid(int row, int col, int num)   // Source: Gemini with
        unchanged logic; Comments by Andy Garcia
    { // Method used to check if a digit can be placed at given
        index by looking through rows, columns, and 3x3 grids
        for (int i = 0; i < 9; ++i)
        { // Loop through every column in the given row
            if (board[row][i] == num)
                return false; // Return false if the number is
        already in the row
        }
        for (int i = 0; i < 9; ++i)
        { // Loop through each row in the current column
            if (board[i][col] == num)
                return false; // Return false if the number is
        already in the col
        }
        int startRow = row - row % 3; // Find the starting row index
        of the 3x3 grid
        int startCol = col - col % 3; // Find the starting column
        index of the 3x3 subgrid
        for (int r = 0; r < 3; ++r)
        { // Loop through all 3 rows of the subgrid
            for (int c = 0; c < 3; ++c)
            { // Loop through all 3 columns of the subgrid
                if (board[startRow + r][startCol + c] == num)
                    return false; // Return false if it is found
            }
        }
        return true; // Return true if the placement is valid across
        row, col, and subgrid
    }

    // Function authored by Andy Garcia to detect if a digit is
        repeated on a row or column. Error detected by Claude and
        given assistance with logic and code implemented by Andy
        Garcia
    bool givensAreValid()
    {                                   // new function to test if
        duplicate nums exist on the same row or col
        for (int r = 0; r < 9; ++r) // loop through every row
```

```
        for (int c = 0; c < 9; ++c)
        {                       // loop through every col
            int d = board[r][c]; // set d equal to the digit at
    the board spot looked at
            if (d == 0)
                continue;               // skip blanks
            board[r][c] = 0;            // take it off the board
            bool ok = isValid(r, c, d); // does it conflict with
    the rest?
            board[r][c] = d;            // put it back
            if (!ok)
                return false; // two givens clash
        }
    return true; // otherwise return true
}

int solutionCount = 0; // new private member added to count the
    number of solutions done

// Code Change #3 : edited Gemini's code to stop storing a
    solution each time it was finished and instead print it off
    then clear it. Storage fix discovered by Claude and code
    implemented by Andy Garcia
void solveRec(int row, int col)  // Source: Gemini; code changed
    by Andy Garcia
{ // Recursive backtracking method. Removed solutions parameter
    (Andy Garcia)
    if (row == 9)
    {                                                       //
    Check if we have successfully processed past the last row
        ++solutionCount;                                    //
    count it
        std::cout << "Solution " << solutionCount << ":\n"; //
    prints what number solution it is on currently
        printBoard();                                       //
    print it now, then discard
        std::cout << "\n";                                  //
    prints new line
        return;                                             //
    ends the function
    }
    int nextRow = (col == 8) ? row + 1 : row; // Calculate the
    row index for the next step of the recursion
    int nextCol = (col == 8) ? 0 : col + 1;   // Calculate the
    column index for the recusion next step

    if (board[row][col] != 0)
    {                           // Checks if cell is already
    filled
        solveRec(nextRow, nextCol); // Go to the next cell
    without changing the current
    }
    else
    { // If the current cell is blank (represented by 0)
        for (int num = 1; num <= 9; ++num)
        { // Try digits 1 through 9 sequentially
            if (isValid(row, col, num))
            {                           // Check if the
    digit in the loop is valid in that space
                board[row][col] = num;      // Temporarily place
    it on the board
                solveRec(nextRow, nextCol); // Use recursion to
    solve the next cell
                board[row][col] = 0;        // Then backtrack
    the cell to be a 0 which represent blank
```

```
                }
            }
        }
    }

public: // Public interface for the SudokuSolver class
    SudokuSolver(const std::vector<std::vector<int>> &initialBoard)
    {                       // Constructor taking initial board
        board = initialBoard; // Initialize the board by using the
        provided puzzle
    }

    int findSolutions()  // Source: Gemini; comments by Andy Garcia
    { // Public method to trigger solving. Updated to take no
        parameters and return an int of how many solutions found
        (Andy Garcia)
        if (!givensAreValid())
            return 0;         // uses new method to quit and have no
        solution if a duplicate is detected (Andy Garcia)
        solveRec(0, 0);       // Start the recursion from the first
        cell which is the index 0,0
        return solutionCount; // returns the solution count when
        findSolutions is called
    }

    void printBoard() const   // Source gemini; comments by Andy
        Garcia
    { // Method to print the current state of the board
        for (int r = 0; r < 9; ++r)
        { // Loop through each row of the board
            for (int c = 0; c < 9; ++c)
            { // Loop through each column of the row
                if (board[r][c] == 0)
                {                       // Check if the cell value is
        0 (blank)
                    std::cout << "_ "; // Print an underscore for
        blank cells
                }
                else
                {                                       // If the cell
        contains a valid digit
                    std::cout << board[r][c] << " "; // Print the
        digit followed by a space
                }
            }
            std::cout << "\n"; // Print a newline at the end of each
        row
        }
    }
};

class PuzzleManager
{       // Build a class to run the program and read the files
public: // These are the public methods for our file management
    void processPuzzle(const std::string &filename)  // source
        Gemini; comments by Andy Garcia
    {
        std::ifstream infile(filename); // Open the file

        std::cout << "File: " << filename << "\n"; // Print the
        current file name
                if (!infile.is_open())
```

```
{
// Check if the file failed to open
    std::cout << "Error: Could not open file " << filename
<< "\n\n"; // Print out the error message
    return;
// Move to the next file
}

std::vector<std::vector<int>> initialBoard(9,
std::vector<int>(9, 0)); // Create 9x9 grid with all zeros
bool readSuccess = true;
// Flag to track successful file parsing
for (int r = 0; r < 9; ++r)
{ // Go through the 9 rows of the grid
    for (int c = 0; c < 9; ++c)
    {           // Go through all 9 columns in each row
        char ch; // Use a variable to hold the character
read in the file
        if (!(infile >> ch))
        {                       // Try to read the file
from the file stream
            readSuccess = false; // Indicate the read was a
failure if unsucssesful
            break;              // Break out of the column
loop
        }
        if (ch == '_')
        {                          // Check if the
character is a blank cell
            initialBoard[r][c] = 0; // Write a 0 to
represent blank cell on our board
        }
        else if (ch >= '1' && ch <= '9')
        {                                  // Check if
character is a digit 1 through 9
            initialBoard[r][c] = ch - '0'; // Change that
character to an int on our board
        }
        else
        {                       // Handle unexpected
characters in the file
            readSuccess = false; // CODE CHANGE #1 : This
detects a bad character and will indicate that to
readSuccess (Idea presented by Claude, code written and
commented by Andy Garcia)
            break;              // leaves the column loop
        }
    }
    if (!readSuccess)
        break; // Break outer loop if reading failed
}

infile.close(); // Close the file stream safely
if (!readSuccess)
{
// Check if puzzle was formatted correctly
    std::cout << "Error: Invalid puzzle in " << filename <<
"\n\n"; //  will now print an error message when a file is
not correct. (Andy Garcia)
    return;
// returns to move on to next file after detecting bad one.
(Andy Garcia)
}
```

```
        SudokuSolver solver(initialBoard);        // one object for
        the whole puzzle

        std::cout << "Original Puzzle:\n";    // prints line before
        original puzzle
        solver.printBoard();                        // prints before
        solving to avoid using two solvers
        int count = solver.findSolutions(); // solutions print as
        they're found
        if (count == 0)
        {                                           // check if there
        were no solutions found   (Andy Garcia)
            std::cout << "No solution found\n\n"; // print no
        solutions found  (Andy Garcia)
        }
        else
        {
            std::cout << "Solutions found: " << count << "\n\n"; //
        print how many solutions were found  (Andy Garcia)
        }
    }

    void run()
    { // Main function to run through all 5 files
        for (int fileIndex = 1; fileIndex <= 5; ++fileIndex)
        {
        // Loops from 1 to 5 to reach all files
            std::string filename = "puzzle" +
        std::to_string(fileIndex) + ".txt"; // build the name of the
        files in a string
            processPuzzle(filename);
        // call new function to shrink run and improve
        maintainability (Andy Garcia)
        }
    }
};

int main()
{                           // Main function of the program
    PuzzleManager manager; // Creates instance of the PuzzleManager
        class
    manager.run();         // Start run() to begin the process
    return 0;              // end the program by returning 0
}
```

This, in its completeness, is the orignal source code before any sort of LLM-related modifications. I will now proceed to do Time and Space complexity analysis on this code. Then, we will choose two large language models to make the small changes required in this Assignment 5 rubric, and discuss how the code individually changes time and space complexity, as well as any other diff that the LLM decides to create. Then, we are going to make changes to improve the code in that regard, and go over those changes. Thanks for joining me on this.

Correctness

Compiling Mr. Garcia's code with g++ yields no compilier warnings or errors on my setup, x86-64-unknown-linux-gnu (v 14.2.1). On the cycle servers Ubuntu (Debian) setup, it also yields no compilier errors or warnings. Therefore, the code is at least compilationally functional.

When executing, it does demonstrate correct answers and solutions, testing against my previous code generated by the Flash I accessed on the website (potentially the same model).

In fact, in pure output, the only difference I can find is that Flash in this instance outputs the actual puzzle before and then after it was solved, while my version only outputs the original file. The solution counts are the same and consistent.

In terms of malformed input, I gave the code some messed up input files. Because of the bizarre constraints not allowing me to include these files in the actual repository, here are the bad input files I used for prosperity:

Too Small Input Size

```
_ _ _ _ _ 4 2 _ 1
_ _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _ 8 3 _ _
_ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4 _ _
_ 7 _ _ 3 5 _ _
```

This has one _ missing. It gave me the output:

```
File: puzzle5.txt Error: Invalid puzzle in puzzle5.txt
```

Handes Newlines

Removing newlines, making the file look like

```
_ _ _ _ _ 4 2 _ 1 _ _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _ 8 3 _ _ _ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4 _ _
_ _ 7 _ _ 3 5 _ _
```

leads the program to output the same solution if puzzle5.txt hand’t had the newlines in the first place. The same is true when adding generous whitespace

```
_ _ _ _ _ 4 2 _ 1
_           _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _       8 3 _ _
_ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4     _ _
_ _ 7 _     _ 3 5 _   _
```

I suspect the internal logic skips over newline chars and space chars, and this makes the implementation this much more robust, so much lauding to Flash and Mr. Garcia.

Too Large Input Size

```
_ _ _ _ _ 4 2 _ 1
_ _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _ 8 3 _ _
_ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4 _ _
_ 7 _ 2 3 5 _ _ _ 3
```

With an extra 3 thrown in the mix, out of bounds. This, unfortunately, leads to the silent error:

```
File: puzzle5.txt
Original Puzzle: _ _ _ _ _ 4 2 _ 1
_ _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _ 8 3 _ _
_ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4 _ _
_ 7 _ 2 3 5 _ _ _
No solution found
```

The buffer doesn't check for any extra characters, meaning any misinput is misinterperted from what it actually is meant to be. Therefore, this program is not entirely correct because it cannot handle all malformed input.

Non-Numerical Character

Adding the char 'A':

```
_ _ _ _ _ 4 2 _ 1
_ _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _ 8 3 _ _
_ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4 _ _
_ 7 _ 2 3 5 _ A _
```

leads to another invalid puzzle error

```
File: puzzle5.txt Error: Invalid puzzle in puzzle5.txt
```

Puzzle File Does Not Exist

Deleting puzzle5.txt and then running the program results in:

```
Error: Could not open file puzzle5.txt
```

Correctness Conclusion

The program compiles fine, and has correct output, but some malformed input files (in my case, having a sudoku file that has more than 81 values) will result in the file being processed up to the 81th element and cutting off the rest, instead of returning an error. This makes the program not completely correct, but is for most usual execution cases completely functional

Time Complexity

This will be rehashing the theoretical more precisely, and then reviewing the actual time complexity as a relationship of the theoretical.

Theoretical

I will rehash the same statement I made previously regarding time complexity within Assignment 4:

Formal Definition

Let A represent our board, which is a matrix of NxN. Subscript notation for a 2D array A[i][j] represent a position at row i and column j, such that A[i][j] represents the (i+j+1) th element in our matrix.

The set A[i] is the set of the row for every value contained within it. Likewise, the set A[j] is set of the column and every value contained within it.

The amount of spots filled is denoted by M.

A sudoku board is consistent iff every ith row has one of the numbers in the range of 1 <= n <= 9. That is, for an iterative set, consistent(A) is such that: * for every i, j in 0 <= i && j <= n-1 * if A[i][j] is in set of A[i] twice * not consistent * if A[i][j] is in set of A[j] twice * not constitent * else * is consistent

Given some board, where N = 9 and M = 80, within our depth first algorithm, it is such that we much check every element within our input search, to check if it is consistent, giving:

9^(81-80) = 9^1 = 9 spots to check

And, to generalize, the formula for any N, M is:

$O(N, M) = N^{(N}2 -M)$

The largest impact term is N^X so the time complexity is N^X, or exponential. Specially for our case, since N is always 9, we can generalized it as.

O(M) = 9^(81-M)

with an inverse relationship… the smaller the M, the larger the time complexity.

Inherent Problem With Depth First Search

Despite our recursion, the algorithm of depth first never checks whether or not we've already pursued a certain path. THis maens that given we have N=3 and M=6

_ 2 _ |
2 1 3 |
_ 3 1 |

Let's say we set A[0]0] to be 1. If we continue with operations, such as A[2][0] = 3, A[0][2] = 2, we found a consistent solution. However, when verifying other solutions, any path can be taken. Our next iteration, where we check A[2][0] and set it to three, ends up resulting in the same non-trivial solution, just with different steps. It is such that we would repeat solutions. In fact, for the number of empty squares, if a solution exists, there is M! ways to repreat this solution. This makes the time complexity very bad, because we do not eliminate duplicates and calculate them, only discarding them when we know they're the same.

Actual

I made my own Python script to manage to measure the real time complexity. I handwrote this, and I leave it here for prosperity:

```
from random import sample
import os, datetime, re, subprocess, time


SOLVABLE_BOARD = """\
5 3 9 8 2 4 1 7 6
1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4
"""

#splits thie input into how i need it
solv_board_formatted = re.split(r'[ \n\s]+', SOLVABLE_BOARD)[:-1]
print(solv_board_formatted)
input()

for i in range(0, 82):
    #build string and write it to file
    f = open("puzzle5.txt", "w")
    solv_board_formatted = re.split(r'[ \n\s]+', SOLVABLE_BOARD)
        [:-1]

    start = time.perf_counter()


# Source - https://stackoverflow.com/a/5597017
# Posted by user355252, modified by community. See post 'Timeline'
        for change history
# Retrieved 2026-10-24, License - CC BY-SA 3.0

    with open(os.devnull, 'wb') as devnull:
```

```
        subprocess.check_call(['./a.out'], stdout=devnull,
        stderr=subprocess.STDOUT)


    for rs in sample(range(81), i):
        solv_board_formatted[rs] = "_"
    f.write("".join(solv_board_formatted))

    #get the time when it started and subtracts by the current time

    elapsed = time.perf_counter() - start

    print(f"diff is {elapsed}")
    open("results.txt", "a").write(f"M = {i} -> {elapsed}\n")
```

The output of my program

```
diff is 0.00545185903320089
diff is 0.0026373660075478256
diff is 0.0021666059619747102
diff is 0.002530121011659503
diff is 0.0021756510250270367
diff is 0.002137768955435604
diff is 0.002133769972715527
diff is 0.0021438749972730875
diff is 0.0020939590176567435
diff is 0.0025313780060969293
diff is 0.002387487969826907
diff is 0.002253651968203485
diff is 0.0022582660312764347
diff is 0.002199187991209328
diff is 0.00244753982566297
diff is 0.0021356380311772227
diff is 0.0024444410228170455
diff is 0.0021521500311791897
diff is 0.002081955026369542
diff is 0.0022210879833437502
diff is 0.002054744982160628
diff is 0.0020828780252486467
diff is 0.0027890809578821063
diff is 0.002232139988336712
diff is 0.0022941739880479872
diff is 0.0022375709959305823
diff is 0.002240996982436627
diff is 0.002228586992714554
diff is 0.0022679029498249292
diff is 0.0022391750244423747
diff is 0.0022088270052336156
diff is 0.002066651009954512
diff is 0.0020616299589164555
diff is 0.0020836680196225643
diff is 0.0020722190383821726
diff is 0.0021858169930055737
diff is 0.0024438260006718338
diff is 0.0022411009995266795
diff is 0.0022055069566704333
diff is 0.0021622059866786003
diff is 0.0021802809787914157
diff is 0.0023885220289230347
diff is 0.0023798600304871798
diff is 0.0023473030305467546
```

```
diff is 0.004238600027747452
diff is 0.0025159650249406695
diff is 0.0037813110393472016
diff is 0.0025767749757505953
diff is 0.005185690009966493
diff is 0.002763491007499397
diff is 0.003756350022740662
diff is 0.01092040300136432
diff is 0.008366678026504815
diff is 0.002676305011846125
diff is 0.013574207026977092
diff is 0.24760583002353087
diff is 0.3609359919792041
diff is 1.0877921709907241
diff is 5.160632780985907
diff is 1.631053916993551
diff is 1.5057089289766736
diff is 11.115237519028597
diff is 9.016612896986771
diff is 13.992875210999046
diff is 715.2867007290479
```

Shows the same expected blow-up. The reason that we have a slightly slower bleed here is there's optimizations being done at the assembly level that give us a boost of speed, even if our algorithm is terrible inefficient. Like most patches in life, it's not enough, and we soon laspe into unreasable solving times for our target N=9.

Space

Theoretical

The theoretical space complexity **should** be O(1), but it's a little more nuanced than that.

It is true that, given our calculation of the solution, all we need to do is recurse downwards with a pass by value. Because of this, we accept a closed input size (N=9, 9x9=81 elements), we preform our validation operation, and then we finish.

However, there are varying ways to achieve this. There was a seperate issue last time, where Opus used significantly more memory than Flash for large M. The reason for this is because Claude stored multiple copies of the solution in a vector that it would pass downwards by value, while Flash would store one global vector by reference passed and allow the recursion to add solutions once found. This is just a practical imperfection, the space complexity should be constant as our input cannot change size and we do not need to store multiple board states because of our setup.

Technically, though, there is a scaling factor on the inputs that we store, especially for large unsolved Sudoku puzzles where there are many solutions. Therefore, since the input scales the space, there is technically a space complexity that is dependent on M. In fact, it tends to grow very quickly, the same as our time complexity. However, it is small and bounded for reasonable and nice M. The worst case scenarion is when M= NxN, or we have a completely empty board. The upper bound is the sizeof each board in memory.

From this paper here: https://www.apprendre-en-ligne.net/blog/docu/sudoku.pdf

- There are N0 = 6670903752021072936960 ≈ 6.671 × 1021 valid Sudoku grids. Taking out the factors of 9! and 722 coming from relabelling and the lexicographical reduction of the top row of blocks B2 and B3, and of the left column of blocks B4 and B7, this leaves 3546146300288 = 27 × 27704267971 arrangements, the last factor being prime

This is our upper bound, and we can know for a fact that our program would take our base memory + board_size * 6.671 x 1021, meaning we could absolutely store all possible answers. If our board is 100 bytes (which is a generous overestimate), we will get approximately:

> > > 100 * 6.671 * 1021 = 681109 bytes ~= 682 kb

This is defintely storable.

Thus, we have a large but finite, though temporally impratical amount of solutions. And, for increasingly smaller M, we will have much less solutions than checks to store. The relationship of this is highly probalistic, and you are welcome to read the paper cited above that discusses calculating possible permutations on more realistic bounds. I will claim that the space complexity is O(M)= e^M for large M, but for small and realistic M is more closely O(M) = 1.

Actual

I had Opus 5.5 generate this script to verify the space complexity of Mr. Garcia's initial program to verify it stays around constant. It's based on my program that I made myself, that I then gave to AI.

```
from random import sample
import os, re

SOLVABLE_BOARD = """\
5 3 9 8 2 4 1 7 6
1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4
"""

RESULTS_FILE = "space_results.txt"

# truncate/create the results file fresh for this run
open(RESULTS_FILE, "w").close()

for i in range(0, 82):
    # start from a fresh copy of the solved board each iteration
    board = re.split(r'[ \n\s]+', SOLVABLE_BOARD)[:-1]
    for rs in sample(range(81), i):
        board[rs] = "_"

    with open("puzzle5.txt", "w") as f:
```

```
        f.write("".join(board))

    with open(os.devnull, "wb") as devnull:
        # Launch the solver ourselves (instead of
        subprocess.check_call) so we
        # can reap it with os.wait4 and read its peak RSS straight
        from the
        # kernel's rusage struct for *this* child only.
        pid = os.fork()
        if pid == 0:
            os.dup2(devnull.fileno(), 1)  # stdout
            os.dup2(devnull.fileno(), 2)  # stderr
            os.execv("./a.out", ["./a.out"])
            os._exit(127)  # only reached if execv fails

        _, status, rusage = os.wait4(pid, 0)

    if os.WIFEXITED(status) and os.WEXITSTATUS(status) != 0:
        print(f"WARNING: a.out exited with status
        {os.WEXITSTATUS(status)} at M={i}")

    # ru_maxrss is in kilobytes on Linux
    max_rss_kb = rusage.ru_maxrss

    print(f"M = {i} -> {max_rss_kb} KB")
    with open(RESULTS_FILE, "a") as rf:
        rf.write(f"M = {i} -> {max_rss_kb} KB\n")
```

Which yielded this output:

```
M = 0 -> 7900 KB M = 1 -> 7904 KB M = 2 -> 7904 KB M = 3 -> 7904 KB
M = 4 -> 7904 KB M = 5 -> 7904 KB M = 6 -> 7904 KB M = 7 -> 7904 KB
M = 8 -> 7904 KB M = 9 -> 7904 KB M = 10 -> 7908 KB M = 11 -> 7908
KB M = 12 -> 7908 KB M = 13 -> 7908 KB M = 14 -> 7908 KB M = 15 ->
7908 KB M = 16 -> 7908 KB M = 17 -> 7908 KB M = 18 -> 7908 KB M = 19
-> 7908 KB M = 20 -> 7908 KB M = 21 -> 7908 KB M = 22 -> 7908 KB M =
23 -> 7908 KB M = 24 -> 7908 KB M = 25 -> 7908 KB M = 26 -> 7908 KB
M = 27 -> 7912 KB M = 28 -> 7912 KB M = 29 -> 7912 KB M = 30 -> 7912
KB
```

Technically, this doesn’t scale linearly. The reason is that more and more memory is used to store longer and longer answers. Wonce we pass the 58 range, the program runs so slowly that I can’t verify the memory stays more or less constant. The theoretical thus tracks fairly closely to the actual time complexity outputted, which slowly increases, like an exponential with a a very widening coefficient.

Maintainability

Before all else, I will say that Mr. Garcia took the commenting every lines recommendation to heart, so much so that it is actually difficult to read the code. However, the comments are very instructful, so I do appreciate it

This program outputted is similar to the previous Flash, which means it inherents a lot of its flaws.

Suffix/Prefix and givensAreValid

As you may remember last time, there was a discussion about how rigid LLM code generated can be. Specifically, we talked about how LLMs tend to do the ridiculous prefix incrementation:

```
    //bool isValid in sudoku solver
        { // Loop through all 3 rows of the subgrid
            for (int c = 0; c < 3; ++c)
```

and would specifically attempt to do prefix incrementation on a for loop where no human would.

Here's another Gemini generated code that does the same thing.

```
        if (row == 9)
        {                                                       //
        Check if we have successfully processed past the last row
            ++solutionCount;                                    //
        count it
```

Doing this incrementation with a prefix is a dead giveaway that an LLM is generating the code. I did an informal test within my Software Engineering lab, and every person who made a for loop did suffix incrementation.

There is one function that is very confusing to me for that reason.

```
// Function authored by Andy Garcia to detect if a digit is repeated
        on a row or column. Error detected by Claude and given
        assistance with logic and code implemented by Andy Garcia

bool givensAreValid()
    {                               // new function to test if
        duplicate nums exist on the same row or col
        for (int r = 0; r < 9; ++r) // loop through every row
            for (int c = 0; c < 9; ++c)
            {                       // loop through every col
                int d = board[r][c]; // set d equal to the digit at
        the board spot looked at
                if (d == 0)
                    continue;               // skip blanks
                board[r][c] = 0;            // take it off the board
                bool ok = isValid(r, c, d); // does it conflict with
        the rest?
                board[r][c] = d;            // put it back
                if (!ok)
                    return false; // two givens clash
            }
        return true; // otherwise return true
    }
```

As you can see on the top, Mr. Garcia says he authored this function. However, the same suffix incrementation is used within the for loop. More than that, we use concise and strange names such as d, c, and r. These are actually the same names that my Flash generated when making this exact function. Lastly, it uses inline if statements that with a tab and enter, something else that Flash is very keen to do as seen here:

```
            if (board[row][i] == num)
                return false; // Return false if the number is
        already in the row
```

So while authored by Mr. Garcia, it seems that it is more LLM generated in at least its syntax and production, giving it the same bad

maintainability quirks.

Unnecessary Complexity

As Tim Peters once said when you run import this in any Python shell.

- Simple is better than complicated. Complicated is better than complex

This is the frame that I felt reviewing some of the changes given that are supposed to patch up the program, but add unnecessary complexity. For example, within main, while reading the file, when we encounter an invalid character/too small input, we set a boolean readSuccess = false and then break. This happens twice, for each case I mentioned respectively.

```
                if (!(infile >> ch))
                {                           // Try to read the file
        from the file stream
                    readSuccess = false; // Indicate the read was a
        failure if unsucssesful
                    break;                  // Break out of the column
        loop
                }


else
                {                           // Handle unexpected
        characters in the file
                    readSuccess = false; // CODE CHANGE #1 : This
        detects a bad character and will indicate that to
        readSuccess (Idea presented by Claude, code written and
        commented by Andy Garcia)
                    break;                  // leaves the column loop
                }
```

Then we check if readSuccess is false:

```
        //this is within our loop
            if (!readSuccess)
                break; // Break outer loop if reading failed
        }
```

and then, once we break out of the loop, wil call a specific error function:

```
        if (!readSuccess)
        {
        // Check if puzzle was formatted correctly
            std::cout << "Error: Invalid puzzle in " << filename <<
        "\n\n"; //  will now print an error message when a file is
        not correct. (Andy Garcia)
            return;
        // returns to move on to next file after detecting bad one.
        (Andy Garcia)
        }
```

But why not just print something and exit? The function and error is not unique, and the same thing could be achieved with

```
if (!(inline >> ch)){std::cout << "Error reading file"; exit(1);}
```

Having a seperate boolean, keeping track of the value, and then erroring and exiting out seems very very redundant. I think Claude working on this attempts to emulate what it would do for large projects, and implements solutions that don't really make sense for small projects.

Namespace Cramping and Overcaution

This is a positive for LLMs on Maintainability, but still work talking about why.

C++, compared to C, has a massive standard library. There are a lot of builtin types that the std library has. For small projects, you can see

```
using namespace std;
```

which allows me to export all types without hierarchy. For example, instead of std::vector, using vector as a namespace.

The issue comes in namespace collision. Because C++ projects are massive, span multiple codebases, and because how C compilies things literally with inclusions (something that C++ somewhat inherents), package managing in C++ is one of the worst experiences known to man. Any redefinition of a namespace in another file will just create an error at best, and at worst an implicit redefinition of what that namespace means. It also becomes very difficult to debug at a high level, when you have millions of C++ files.

Moreover, C++ std names can be generic enough that they could be implemented by libraries before C++ did it themselves. Most companies transition to the next version of C++ very slowly, partly for this reason (most companies currently run C++11 or, if they're feeling risky, C++17). This can cause so much unintended problems that it is much better to use std:: explicitly, even if it is verbose.

In such a small project like this, it is really clear that there would be no namespace collision, because there isn't even a second file. But what if that changed? Maintainability wise, we're always looking for the best future proof solution. I think that Flash did a good job at future proofing, even if here it's a little explicit.

Prompt and LLMs

As per the rubric, I will be selecting two LLMs to make the minor changes decribed in the rubric.

The changes, which I keep here as the prompt I used, are as follows:

```
Modify the following C++ program attached with these
features/changes:


Write the solutions for each puzzle file to a file named
solutionN.txt (N being the puzzle number)

    * If there's no solutions, display "No solution found"
    * If solutions are found, it should **all** solutions followed
```

```
by a single line "Solutions found: N" where N is the number of
solutions
    * These should be in the same format as the puzzle1.txt file
I've attached
```

Attached to my prompt was Mr. Garcia's unmodifed Assignment4.cpp, and then puzzle1.txt.

I firstly used Flash, because I was curious how it would modify its own code. I accessed Flash by going to https://www.gemini.google.com, and logged into my Google Account to allow for file uploading (small aside, I recently saw an ad about how you don't need to sign in anymore to use Gemini, which is telling on how the AI race has been about how throwaway and easy the free models are to access, but I'm disgressing).

For my second model, I decided to give Astra a try. When I used OpenAI's ChatGPT website back in September, GPT 6.0 had just been launched and free users only had access to Luna or Sol, but now with Astra access I was curious on what the update would implement.

Here are both of their outputs, respectively in mention order:

Gemini Flash (Medium)

```
/*
EECS 348 Assignment 4
Sudoku Puzzle Solver
Description - Solves the sudoku puzzle using a recursive depth-first
        search
with backtracking. Finds all solutions for each puzzle or reports if
        none exist,
writing the results to solutionN.txt files.

Inputs - The five given txt puzzle files. Each have a 9x9 grid with
        (_) being a
blank cell

Outputs - Each puzzle outputs the file name, original puzzle, and
        writes either
every solution found or "No solution found" to solutionN.txt

Collaborators - n/a
Other Sources - Base code generated by Google Gemini via the
        website. Claude used to assist
with design changes by presenting ideas all code adjustments written
        by Andy Garcia. All comments rewritten by Andy Garcia

Author - Andy Garcia

Creation Date - 10/14/26
Revision Date - 10/27/26
*/

#include <iostream> // Include input-output stream library for
        console printing
#include <fstream>  // Include file stream library for reading and
        writing puzzle text files
#include <vector>   // Include vector library to store the 9x9 grid
        and list of solutions
#include <string>   // Include string library for handling file
        names and strings
```

```cpp
class SudokuSolver
{                                       // Creates a class to make
        a SudokuSolver
private:                                // Make it private instead
        of public
    std::vector<std::vector<int>> board; // Use a 2d vector to
        represent the grid of size 9x9
    int solutionCount = 0;              // Private member added to
        count the number of solutions found
    std::ostream* outStream = &std::cout;// Stream pointer for
        output (console or file)

    bool isValid(int row, int col, int num)   // Source: Gemini with
        unchanged logic; Comments by Andy Garcia
    { // Method used to check if a digit can be placed at given
        index by looking through rows, columns, and 3x3 grids
        for (int i = 0; i < 9; ++i)
        { // Loop through every column in the given row
            if (board[row][i] == num)
                return false; // Return false if the number is
        already in the row
        }
        for (int i = 0; i < 9; ++i)
        { // Loop through each row in the current column
            if (board[i][col] == num)
                return false; // Return false if the number is
        already in the col
        }
        int startRow = row - row % 3; // Find the starting row index
        of the 3x3 grid
        int startCol = col - col % 3; // Find the starting column
        index of the 3x3 subgrid
        for (int r = 0; r < 3; ++r)
        { // Loop through all 3 rows of the subgrid
            for (int c = 0; c < 3; ++c)
            { // Loop through all 3 columns of the subgrid
                if (board[startRow + r][startCol + c] == num)
                    return false; // Return false if it is found
            }
        }
        return true; // Return true if the placement is valid across
        row, col, and subgrid
    }

    // Function authored by Andy Garcia to detect if a digit is
        repeated on a row or column. Error detected by Claude and
        given assistance with logic and code implemented by Andy
        Garcia
    bool givensAreValid()
    {                                   // new function to test if
        duplicate nums exist on the same row or col
        for (int r = 0; r < 9; ++r) // loop through every row
            for (int c = 0; c < 9; ++c)
            {                           // loop through every col
                int d = board[r][c]; // set d equal to the digit at
        the board spot looked at
                if (d == 0)
                    continue;               // skip blanks
                board[r][c] = 0;            // take it off the board
                bool ok = isValid(r, c, d); // does it conflict with
        the rest?
                board[r][c] = d;            // put it back
                if (!ok)
```

```
                    return false; // two givens clash
                }
            return true; // otherwise return true
        }

        // Code Change #3 : edited Gemini's code to stop storing a
            solution each time it was finished and instead print it off
            then clear it. Storage fix discovered by Claude and code
            implemented by Andy Garcia
        void solveRec(int row, int col)  // Source: Gemini; code changed
            by Andy Garcia
        { // Recursive backtracking method. Removed solutions parameter
            (Andy Garcia)
            if (row == 9)
            {                                   // Check if we have
            successfully processed past the last row
                ++solutionCount;                // count it
                printBoard(*outStream);         // print the solution
            board to the specified stream
                *outStream << "\n";             // prints blank line
            between solutions for readability
                return;                         // ends the function
            }
            int nextRow = (col == 8) ? row + 1 : row; // Calculate the
            row index for the next step of the recursion
            int nextCol = (col == 8) ? 0 : col + 1;   // Calculate the
            column index for the recursion next step

            if (board[row][col] != 0)
            {                                 // Checks if cell is already
            filled
                solveRec(nextRow, nextCol); // Go to the next cell
            without changing the current
            }
            else
            { // If the current cell is blank (represented by 0)
                for (int num = 1; num <= 9; ++num)
                { // Try digits 1 through 9 sequentially
                    if (isValid(row, col, num))
                    {                               // Check if the
            digit in the loop is valid in that space
                        board[row][col] = num;      // Temporarily place
            it on the board
                        solveRec(nextRow, nextCol); // Use recursion to
            solve the next cell
                        board[row][col] = 0;        // Then backtrack
            the cell to be a 0 which represent blank
                    }
                }
            }
        }

    public: // Public interface for the SudokuSolver class
        SudokuSolver(const std::vector<std::vector<int>> &initialBoard)
        {                           // Constructor taking initial board
            board = initialBoard; // Initialize the board by using the
            provided puzzle
        }

        int findSolutions(std::ostream &stream = std::cout)  // Source:
            Gemini; comments by Andy Garcia
        { // Public method to trigger solving with an output stream
            parameter (Andy Garcia)
            outStream = &stream;  // Set the output stream pointer
```

```
        if (!givensAreValid())
            return 0;         // uses new method to quit and have no
        solution if a duplicate is detected (Andy Garcia)
        solveRec(0, 0);       // Start the recursion from the first
        cell which is the index 0,0
        return solutionCount; // returns the solution count when
        findSolutions is called
    }

    void printBoard(std::ostream &out = std::cout) const   // Source
        gemini; comments by Andy Garcia
    { // Method to print the current state of the board to a given
        stream
        for (int r = 0; r < 9; ++r)
        { // Loop through each row of the board
            for (int c = 0; c < 9; ++c)
            { // Loop through each column of the row
                if (board[r][c] == 0)
                {                       // Check if the cell value is
        0 (blank)
                    out << "_ ";       // Print an underscore for
        blank cells
                }
                else
                {                                   // If the cell
        contains a valid digit
                    out << board[r][c] << " "; // Print the digit
        followed by a space
                }
            }
            out << "\n"; // Print a newline at the end of each row
        }
    }
};

class PuzzleManager
{       // Build a class to run the program and read the files
public: // These are the public methods for our file management
    void processPuzzle(const std::string &filename, int fileIndex)
        // source Gemini; comments by Andy Garcia
    {
        std::ifstream infile(filename); // Open the puzzle input
        file
        std::string solFilename = "solution" +
        std::to_string(fileIndex) + ".txt";
        std::ofstream outfile(solFilename); // Open the solution
        output file

        std::cout << "File: " << filename << "\n"; // Print the
        current file name
        if (!infile.is_open())
        {
        // Check if the file failed to open
            std::cout << "Error: Could not open file " << filename
        << "\n\n"; // Print out the error message
            if (outfile.is_open()) {
                outfile << "Error: Could not open file " << filename
        << "\n";
                outfile.close();
            }
            return;
        // Move to the next file
        }
```

```
std::vector<std::vector<int>> initialBoard(9,
std::vector<int>(9, 0)); // Create 9x9 grid with all zeros
bool readSuccess = true;
// Flag to track successful file parsing
for (int r = 0; r < 9; ++r)
{ // Go through the 9 rows of the grid
    for (int c = 0; c < 9; ++c)
    {           // Go through all 9 columns in each row
        char ch; // Use a variable to hold the character
read in the file
        if (!(infile >> ch))
        {                       // Try to read the file
from the file stream
            readSuccess = false; // Indicate the read was a
failure if unsuccessful
            break;              // Break out of the column
loop
        }
        if (ch == '_')
        {                          // Check if the
character is a blank cell
            initialBoard[r][c] = 0; // Write a 0 to
represent blank cell on our board
        }
        else if (ch >= '1' && ch <= '9')
        {                                  // Check if
character is a digit 1 through 9
            initialBoard[r][c] = ch - '0'; // Change that
character to an int on our board
        }
        else
        {                       // Handle unexpected
characters in the file
            readSuccess = false; // CODE CHANGE #1 : This
detects a bad character and will indicate that to
readSuccess (Idea presented by Claude, code written and
commented by Andy Garcia)
            break;              // leaves the column loop
        }
    }
    if (!readSuccess)
        break; // Break outer loop if reading failed
}

infile.close(); // Close the input file stream safely
if (!readSuccess)
{
// Check if puzzle was formatted correctly
    std::cout << "Error: Invalid puzzle in " << filename <<
"\n\n"; // will now print an error message when a file is
not correct. (Andy Garcia)
    if (outfile.is_open()) {
        outfile << "Error: Invalid puzzle in " << filename
<< "\n";
        outfile.close();
    }
    return;
// returns to move on to next file after detecting bad one.
(Andy Garcia)
}

SudokuSolver solver(initialBoard);        // create solver
object for the puzzle
```

```
        std::cout << "Original Puzzle:\n";    // prints line before
        original puzzle
        solver.printBoard(std::cout);             // prints original
        board to console

        int count = solver.findSolutions(outfile); // find solutions
        and write them to solutionN.txt

        if (outfile.is_open())
        {
            if (count == 0)
            {
                outfile << "No solution found\n";
            }
            else
            {
                outfile << "Solutions found: " << count << "\n";
            }
            outfile.close(); // Close solution file stream
        }

        if (count == 0)
        {                                         // check if there
        were no solutions found   (Andy Garcia)
            std::cout << "No solution found\n\n"; // print no
        solutions found  (Andy Garcia)
        }
        else
        {
            std::cout << "Solutions found: " << count << "\n\n"; //
        print how many solutions were found  (Andy Garcia)
        }
        std::cout << "Solutions written to " << solFilename <<
        "\n\n";
    }

    void run()
    { // Main function to run through all 5 files
        for (int fileIndex = 1; fileIndex <= 5; ++fileIndex)
        {
        // Loops from 1 to 5 to reach all files
            std::string filename = "puzzle" +
        std::to_string(fileIndex) + ".txt"; // build the name of the
        files in a string
            processPuzzle(filename, fileIndex);
        // call processPuzzle with filename and index (Andy Garcia)
        }
    }
};

int main()
{                          // Main function of the program
    PuzzleManager manager; // Creates instance of the PuzzleManager
        class
    manager.run();         // Start run() to begin the process
    return 0;              // end the program by returning 0
}

/*
EECS 348 Assignment 4
Sudoku Puzzle Solver
Description - Solves the sudoku puzzle using a recursive depth-first
        search
```

```
with backtracking. Finds all solutions for each puzzle or reports if
        none exist

Inputs - The five given txt puzzle files. Each have a 9x9 grid with
        (_) being a
blank cell

Outputs - Each puzzle outputs the file name, the original puzzle,
        and either every solution found or "no solution found"

Collaborators - n/a
Other Sources - Base code generated by Google Gemini via the
        website. Claude used to assist
with design changes by presenting ideas all code adjustments written
        by Andy Garcia. All comments rewritten by Andy Garcia

Author - Andy Garcia

Creation Date - 10/14/26
Revision Date -
*/

#include <iostream> // Include input-output stream library for
        console printing
#include <fstream>  // Include file stream library for reading
        puzzle text files
#include <vector>   // Include vector library to store the 9x9 grid
        and list of solutions
#include <string>   // Include string library for handling file
        names and strings

class SudokuSolver
{                                       // Creates a class to make
        a SudokuSolver
private:                                // Make it private instead
        of public
    std::vector<std::vector<int>> board; // Use a 2d vector to
        represent the grid of size 9x9

    bool isValid(int row, int col, int num)   // Source: Gemini with
        unchanged logic; Comments by Andy Garcia
    { // Method used to check if a digit can be placed at given
        index by looking through rows, columns, and 3x3 grids
        for (int i = 0; i < 9; ++i)
        { // Loop through every column in the given row
            if (board[row][i] == num)
                return false; // Return false if the number is
        already in the row
        }
        for (int i = 0; i < 9; ++i)
        { // Loop through each row in the current column
            if (board[i][col] == num)
                return false; // Return false if the number is
        already in the col
        }
        int startRow = row - row % 3; // Find the starting row index
        of the 3x3 grid
        int startCol = col - col % 3; // Find the starting column
        index of the 3x3 subgrid
        for (int r = 0; r < 3; ++r)
        { // Loop through all 3 rows of the subgrid
            for (int c = 0; c < 3; ++c)
            { // Loop through all 3 columns of the subgrid
                if (board[startRow + r][startCol + c] == num)
```

```
                return false; // Return false if it is found
            }
        }
        return true; // Return true if the placement is valid across
        row, col, and subgrid
    }

    // Function authored by Andy Garcia to detect if a digit is
        repeated on a row or column. Error detected by Claude and
        given assistance with logic and code implemented by Andy
        Garcia
    bool givensAreValid()
    {                               // new function to test if
        duplicate nums exist on the same row or col
        for (int r = 0; r < 9; ++r) // loop through every row
            for (int c = 0; c < 9; ++c)
            {                        // loop through every col
                int d = board[r][c]; // set d equal to the digit at
        the board spot looked at
                if (d == 0)
                    continue;               // skip blanks
                board[r][c] = 0;            // take it off the board
                bool ok = isValid(r, c, d); // does it conflict with
        the rest?
                board[r][c] = d;            // put it back
                if (!ok)
                    return false; // two givens clash
            }
        return true; // otherwise return true
    }

    int solutionCount = 0; // new private member added to count the
        number of solutions done

    // Code Change #3 : edited Gemini's code to stop storing a
        solution each time it was finished and instead print it off
        then clear it. Storage fix discovered by Claude and code
        implemented by Andy Garcia
    void solveRec(int row, int col, std::ostream &output)  // Write
        each solution to the output file
    { // Recursive backtracking method. Removed solutions parameter
        (Andy Garcia)
        if (row == 9)
        {                                                       //
        Check if we have successfully processed past the last row
            ++solutionCount;                                    //
        count it
            output << "Solution " << solutionCount << ":\n"; //
        Label each solution
            printBoard(output);                                  //
        Write the solved board
            output << "\n";                                     //
        Separate solutions
            return;                                             //
        ends the function
        }
        int nextRow = (col == 8) ? row + 1 : row; // Calculate the
        row index for the next step of the recursion
        int nextCol = (col == 8) ? 0 : col + 1;   // Calculate the
        column index for the recusion next step

        if (board[row][col] != 0)
        {                               // Checks if cell is already
        filled
```

```
            solveRec(nextRow, nextCol, output); // Go to the next
        cell without changing the current
        }
        else
        { // If the current cell is blank (represented by 0)
            for (int num = 1; num <= 9; ++num)
            { // Try digits 1 through 9 sequentially
                if (isValid(row, col, num))
                {                               // Check if the
        digit in the loop is valid in that space
                    board[row][col] = num;      // Temporarily place
        it on the board
                    solveRec(nextRow, nextCol, output); // Use
        recursion to solve the next cell
                    board[row][col] = 0;        // Then backtrack
        the cell to be a 0 which represent blank
                }
            }
        }
    }

public: // Public interface for the SudokuSolver class
    SudokuSolver(const std::vector<std::vector<int>> &initialBoard)
    {                           // Constructor taking initial board
        board = initialBoard; // Initialize the board by using the
        provided puzzle
    }

    int findSolutions(std::ostream &output)  // Find and write all
        solutions
    { // Public method to trigger solving. Updated to take no
        parameters and return an int of how many solutions found
        (Andy Garcia)
        if (!givensAreValid())
            return 0;         // uses new method to quit and have no
        solution if a duplicate is detected (Andy Garcia)
        solveRec(0, 0, output); // Start the recursion from the
        first cell
        return solutionCount; // returns the solution count when
        findSolutions is called
    }

    void printBoard(std::ostream &output = std::cout) const   //
        Print the current board to a stream
    { // Method to print the current state of the board
        for (int r = 0; r < 9; ++r)
        { // Loop through each row of the board
            for (int c = 0; c < 9; ++c)
            { // Loop through each column of the row
                if (board[r][c] == 0)
                {                       // Check if the cell value is
        0 (blank)
                    output << "_ "; // Print an underscore for blank
        cells
                }
                else
                {                                   // If the cell
        contains a valid digit
                    output << board[r][c] << " "; // Print the digit
        followed by a space
                }
            }
            output << "\n"; // Print a newline at the end of each
        row
```

```
        }
    }
};

class PuzzleManager
{       // Build a class to run the program and read the files
public: // These are the public methods for our file management
    void processPuzzle(const std::string &filename)  // source
        Gemini; comments by Andy Garcia
    {
        std::ifstream infile(filename); // Open the file

        std::cout << "File: " << filename << "\n"; // Print the
        current file name
                if (!infile.is_open())
        {
        // Check if the file failed to open
            std::cout << "Error: Could not open file " << filename
        << "\n\n"; // Print out the error message
            return;
        // Move to the next file
        }

        std::vector<std::vector<int>> initialBoard(9,
        std::vector<int>(9, 0)); // Create 9x9 grid with all zeros
        bool readSuccess = true;
        // Flag to track successful file parsing
        for (int r = 0; r < 9; ++r)
        { // Go through the 9 rows of the grid
            for (int c = 0; c < 9; ++c)
            {           // Go through all 9 columns in each row
                char ch; // Use a variable to hold the character
        read in the file
                if (!(infile >> ch))
                {                       // Try to read the file
        from the file stream
                    readSuccess = false; // Indicate the read was a
        failure if unsucssesful
                    break;              // Break out of the column
        loop
                }
                if (ch == '_')
                {                          // Check if the
        character is a blank cell
                    initialBoard[r][c] = 0; // Write a 0 to
        represent blank cell on our board
                }
                else if (ch >= '1' && ch <= '9')
                {                                  // Check if
        character is a digit 1 through 9
                    initialBoard[r][c] = ch - '0'; // Change that
        character to an int on our board
                }
                else
                {                       // Handle unexpected
        characters in the file
                    readSuccess = false; // CODE CHANGE #1 : This
        detects a bad character and will indicate that to
        readSuccess (Idea presented by Claude, code written and
        commented by Andy Garcia)
                    break;              // leaves the column loop
                }
            }
            if (!readSuccess)
```

```
                break; // Break outer loop if reading failed
            }

            infile.close(); // Close the file stream safely
            if (!readSuccess)
            {
            // Check if puzzle was formatted correctly
                std::cout << "Error: Invalid puzzle in " << filename <<
            "\n\n"; //  will now print an error message when a file is
            not correct. (Andy Garcia)
                return;
            // returns to move on to next file after detecting bad one.
            (Andy Garcia)
            }

            // Convert puzzleN.txt to solutionN.txt.
            std::string solutionFilename = filename;
            solutionFilename.replace(0, std::string("puzzle").size(),
            "solution");
            std::ofstream solutionFile(solutionFilename);
            if (!solutionFile.is_open())
            {
                std::cout << "Error: Could not create file " <<
            solutionFilename << "\n\n";
                return;
            }

            SudokuSolver solver(initialBoard);
            int count = solver.findSolutions(solutionFile);
            if (count == 0)
            {
                solutionFile << "No solution found\n";
                std::cout << "No solution found. See " <<
            solutionFilename << "\n\n";
            }
            else
            {
                solutionFile << "Solutions found: " << count << "\n";
                std::cout << "Solutions found: " << count << ". See " <<
            solutionFilename << "\n\n";
            }
            solutionFile.close();
        }

        void run()
        { // Main function to run through all 5 files
            for (int fileIndex = 1; fileIndex <= 5; ++fileIndex)
            {
            // Loops from 1 to 5 to reach all files
                std::string filename = "puzzle" +
            std::to_string(fileIndex) + ".txt"; // build the name of the
            files in a string
                processPuzzle(filename);
            // call new function to shrink run and improve
            maintainability (Andy Garcia)
            }
        }
    };

    int main()
    {                          // Main function of the program
        PuzzleManager manager; // Creates instance of the PuzzleManager
            class
```

```
    manager.run();          // Start run() to begin the process
    return 0;               // end the program by returning 0
}
```

Now we will do further analysis on the diff between the two files, and talk about how much it impacts the previous four points from Mr. Garcia's code

Second Analysis

We can see an exact history of the changes by using diff in Linux. Below is Gemini Flash's and then OpenAI's diff:

```
diff Assignment4.cpp  google.c
pp
5c5,6
< with backtracking. Finds all solutions for each puzzle or reports
 if none exist
---
> with backtracking. Finds all solutions for each puzzle or reports
 if none exist,
> writing the results to solutionN.txt files.
10c11,12
< Outputs - Each puzzle outputs the file name, the original puzzle,
 and either every solution found or "no solution found"
---
> Outputs - Each puzzle outputs the file name, original puzzle, and
 writes either
> every solution found or "No solution found" to solutionN.txt
19c21
< Revision Date -
---
> Revision Date - 10/27/26
23c25
< #include <fstream>  // Include file stream library for reading pu
zzle text files
---
> #include <fstream>  // Include file stream library for reading an
d writing puzzle text files
30a33,34
>     int solutionCount = 0;               // Private member added
to count the number of solutions found
>     std::ostream* outStream = &std::cout;// Stream pointer for ou
tput (console or file)
58d61
<
76,77d78
<     int solutionCount = 0; // new private member added to count t
he number of solutions done
<
82,87c83,87
<         {                                                       /
/ Check if we have successfully processed past the last row
<             ++solutionCount;                                    /
/ count it
<             std::cout << "Solution " << solutionCount << ":\n"; /
/ prints what number solution it is on currently
<             printBoard();                                       /
/ print it now, then discard
```

```
<             std::cout << "\n";                                 /
/ prints new line
<             return;                                           /
/ ends the function
---
>         {                                   // Check if we have s
uccessfully processed past the last row
>             ++solutionCount;                // count it
>             printBoard(*outStream);         // print the solution
 board to the specified stream
>             *outStream << "\n";             // prints blank line
between solutions for readability
>             return;                         // ends the function
90c90
<         int nextCol = (col == 8) ? 0 : col + 1;   // Calculate th
e column index for the recusion next step
---
>         int nextCol = (col == 8) ? 0 : col + 1;   // Calculate th
e column index for the recursion next step
116,117c116,118
<     int findSolutions()  // Source: Gemini; comments by Andy Garc
ia
<     { // Public method to trigger solving. Updated to take no par
ameters and return an int of how many solutions found (Andy Garcia)
---
>     int findSolutions(std::ostream &stream = std::cout)  // Sourc
e: Gemini; comments by Andy Garcia
>     { // Public method to trigger solving with an output stream p
arameter (Andy Garcia)
>         outStream = &stream;  // Set the output stream pointer
124,125c125,126
<     void printBoard() const   // Source gemini; comments by Andy
Garcia
<     { // Method to print the current state of the board
---
>     void printBoard(std::ostream &out = std::cout) const   // Sou
rce gemini; comments by Andy Garcia
>     { // Method to print the current state of the board to a give
n stream
132c133
<                     std::cout << "_ "; // Print an underscore for
 blank cells
---
>                     out << "_ ";       // Print an underscore for
 blank cells
136c137
<                     std::cout << board[r][c] << " "; // Print the
 digit followed by a space
---
>                     out << board[r][c] << " "; // Print the digit
 followed by a space
139c140
<             std::cout << "\n"; // Print a newline at the end of e
ach row
---
>             out << "\n"; // Print a newline at the end of each ro
w
147c148
<     void processPuzzle(const std::string &filename)  // source Ge
mini; comments by Andy Garcia
```

```
---
>     void processPuzzle(const std::string &filename, int fileIndex
)  // source Gemini; comments by Andy Garcia
149c150,152
<         std::ifstream infile(filename); // Open the file
---
>         std::ifstream infile(filename); // Open the puzzle input
file
>         std::string solFilename = "solution" + std::to_string(fil
eIndex) + ".txt";
>         std::ofstream outfile(solFilename); // Open the solution
output file
152c155
<                 if (!infile.is_open())
---
>         if (!infile.is_open())
154a158,161
>             if (outfile.is_open()) {
>                 outfile << "Error: Could not open file " << filen
ame << "\n";
>                 outfile.close();
>             }
167c174
<                     readSuccess = false; // Indicate the read was
 a failure if unsucssesful
---
>                     readSuccess = false; // Indicate the read was
 a failure if unsuccessful
188c195
<         infile.close(); // Close the file stream safely
---
>         infile.close(); // Close the input file stream safely
191c198,202
<             std::cout << "Error: Invalid puzzle in " << filename
<< "\n\n"; //  will now print an error message when a file is not c
orrect. (Andy Garcia)
---
>             std::cout << "Error: Invalid puzzle in " << filename
<< "\n\n"; // will now print an error message when a file is not co
rrect. (Andy Garcia)
>             if (outfile.is_open()) {
>                 outfile << "Error: Invalid puzzle in " << filenam
e << "\n";
>                 outfile.close();
>             }
195c206
<         SudokuSolver solver(initialBoard);        // one object f
or the whole puzzle
---
>         SudokuSolver solver(initialBoard);        // create solve
r object for the puzzle
198,199c209,225
<         solver.printBoard();                      // prints befor
e solving to avoid using two solvers
<         int count = solver.findSolutions(); // solutions print as
 they're found
---
>         solver.printBoard(std::cout);             // prints origi
nal board to console
>
```

```
>         int count = solver.findSolutions(outfile); // find soluti
ons and write them to solutionN.txt
>
>         if (outfile.is_open())
>         {
>             if (count == 0)
>             {
>                 outfile << "No solution found\n";
>             }
>             else
>             {
>                 outfile << "Solutions found: " << count << "\n"
>             }
>             outfile.close(); // Close solution file stream
>         }
>
206c232
<             std::cout << "Solutions found: " << count << "\n\n"
//   print how many solutions were found  (Andy Garcia)
---
>             std::cout << "Solutions found: " << count << "\n\n"
// print how many solutions were found  (Andy Garcia)
207a234
>         std::cout << "Solutions written to " << solFilename <<
n\n";
215c242
<             processPuzzle(filename);
                // call new function to shrink run and improve m
ntainability (Andy Garcia)
---
>             processPuzzle(filename, fileIndex);
                // call processPuzzle with filename and index (A
y Garcia)
225c252
< }
---
> }
\ No newline at end of file
l382f965@cycle3:~/Code/SoftwareEng/Fall2026_EECS_348_Assignment_5
/
```

And then for GPT 6, Astra:

```
diff Assignment4.cpp openai.cp
p
58d57
<
79c78
<     void solveRec(int row, int col)  // Source: Gemini; code chan
ged by Andy Garcia
---
>     void solveRec(int row, int col, std::ostream &output)  // Wri
te each solution to the output file
84,86c83,85
<             std::cout << "Solution " << solutionCount << ":\n"; /
/ prints what number solution it is on currently
<             printBoard();                                        /
/ print it now, then discard
<             std::cout << "\n";                                   /
/ prints new line
---
```

```
>             output << "Solution " << solutionCount << ":\n"; // L
abel each solution
>             printBoard(output);
// Write the solved board
>             output << "\n";                                     /
/ Separate solutions
94c93
<             solveRec(nextRow, nextCol); // Go to the next cell wi
thout changing the current
---
>             solveRec(nextRow, nextCol, output); // Go to the next
 cell without changing the current
103c102
<                     solveRec(nextRow, nextCol); // Use recursion
to solve the next cell
---
>                     solveRec(nextRow, nextCol, output); // Use re
cursion to solve the next cell
116c115
<     int findSolutions()  // Source: Gemini; comments by Andy Garc
ia
---
>     int findSolutions(std::ostream &output)  // Find and write al
l solutions
120c119
<         solveRec(0, 0);       // Start the recursion from the fir
st cell which is the index 0,0
---
>         solveRec(0, 0, output); // Start the recursion from the f
irst cell
124c123
<     void printBoard() const   // Source gemini; comments by Andy
Garcia
---
>     void printBoard(std::ostream &output = std::cout) const   //
Print the current board to a stream
132c131
<                     std::cout << "_ "; // Print an underscore for
 blank cells
---
>                     output << "_ "; // Print an underscore for bl
ank cells
136c135
<                     std::cout << board[r][c] << " "; // Print the
 digit followed by a space
---
>                     output << board[r][c] << " "; // Print the di
git followed by a space
139c138
<             std::cout << "\n"; // Print a newline at the end of e
ach row
---
>             output << "\n"; // Print a newline at the end of each
 row
195c194,202
<         SudokuSolver solver(initialBoard);        // one object f
or the whole puzzle
---
>         // Convert puzzleN.txt to solutionN.txt.
>         std::string solutionFilename = filename;
```

```
>         solutionFilename.replace(0, std::string("puzzle").size(),
 "solution");
>         std::ofstream solutionFile(solutionFilename);
>         if (!solutionFile.is_open())
>         {
>             std::cout << "Error: Could not create file " << solut
ionFilename << "\n\n";
>             return;
>         }
197,199c204,205
<         std::cout << "Original Puzzle:\n";    // prints line befo
re original puzzle
<         solver.printBoard();                      // prints befor
e solving to avoid using two solvers
<         int count = solver.findSolutions(); // solutions print as
 they're found
---
>         SudokuSolver solver(initialBoard);
>         int count = solver.findSolutions(solutionFile);
201,202c207,209
<         {                                         // check if the
re were no solutions found   (Andy Garcia)
<             std::cout << "No solution found\n\n"; // print no sol
utions found  (Andy Garcia)
---
>         {
>             solutionFile << "No solution found\n";
>             std::cout << "No solution found. See " << solutionFil
ename << "\n\n";
206c213,214
<             std::cout << "Solutions found: " << count << "\n\n";
//   print how many solutions were found  (Andy Garcia)
---
>             solutionFile << "Solutions found: " << count << "\n";
>             std::cout << "Solutions found: " << count << ". See "
 << solutionFilename << "\n\n";
207a216
>         solutionFile.close();
l
```

Not only are the diffs fairly small, several of them are just changing extra characters. This makes our diff from Linux not even show the whole picture

Correctness - Changes

Correctness is how stricly they followed the instructions. Flash and Astra did this pretty well, it was hard to mess up. However, I think I'll take more point off of Flash

Specifically, I specified that the file output should be within the file:

```
Write the solutions for each puzzle file to a file named
solutionN.txt (N being the puzzle number)
```

And while Flash did output this correctly:

```
(solution3.txt)
7 8 5 3 2 6 1 4 9
```

```
3 4 2 5 1 9 6 7 8
6 1 9 8 4 7 5 2 3
4 9 3 6 8 5 2 1 7
1 2 8 9 7 4 3 6 5
5 7 6 1 3 2 8 9 4
2 5 1 4 9 3 7 8 6
9 3 7 2 6 8 4 5 1
8 6 4 7 5 1 9 3 2

Solutions found: 1
```

it also did normal stdout output like it did previously. It just also so happens that it piped the output out.

Astra did the same output

```
(solution3.txt)
Solution 1:
7 8 5 3 2 6 1 4 9
3 4 2 5 1 9 6 7 8
6 1 9 8 4 7 5 2 3
4 9 3 6 8 5 2 1 7
1 2 8 9 7 4 3 6 5
5 7 6 1 3 2 8 9 4
2 5 1 4 9 3 7 8 6
9 3 7 2 6 8 4 5 1
8 6 4 7 5 1 9 3 2

Solutions found: 1
```

But has different stdout output, that just hints at what the solution is.

```
File: puzzle1.txt Solutions found: 1. See solution1.txt
File: puzzle2.txt Solutions found: 1. See solution2.txt
File: puzzle3.txt Solutions found: 1. See solution3.txt
File: puzzle4.txt No solution found. See solution4.txt
File: puzzle5.txt Solutions found: 2. See solution5.txt
```

So I will give Astra a better score because it actually writes to just the file and not the screen. Though, functionally, I didn't say not to print it, so both get full marks on there.

Time Complexity

Both of these programs are not changed in time and space complexity by either LLM. Their core functionality remains untouched– they implement the same algorithm, just writing output to a file. Therefore, this section remains. All instructions to write are linear and do not scale with input more than the display already does:

```
//google flash's SudokuSolver, line 215
           if (count == 0)
            {
                outfile << "No solution found\n";
            }
            else
            {
                outfile << "Solutions found: " << count << "\n";
            }
            outfile.close(); // Close solution file stream
```

```
        }
```

for example, is scaled with the display output.

Space Complexity

Space complexity also does not change. SPace complexity is the amount the program changes within its space requrements while executing, and the program does not require extra space. In fact, we do not store the buffer that we even write to, as it is all handled by the operating system when we write and displa output. Therefore, we have no space complexity changes within our program, and the same criticisms as before remained just as valid.

Maintainability

Both Flash and Astra use ofstream to write to our file

```
//openai, line 197
std::ofstream solutionFile(solutionFilename);

//google, line 152
std::ofstream outfile(solFilename); // Open the solution output file
```

These two are alright, and the standard way. Instead of then just displaying the output, they write by using streams to the file instead of the display. We pass the ofstream by reference in Flash

```
int findSolutions(std::ostream &stream = std::cout)
```

and also the same in Astra

```
//line 115, openai
    int findSolutions(std::ostream &output)  // Find and write all
        solutions
```

and we just replace every instead where we would write to cout with output. That's it, there's no extra frills. In terms of maintainability, it is hard to critique anything, because there is no real changes. It avoids namespace colliding, it handles memory safely through reference instead of making a new ofstream object each time, and it is simple. This is nothing but an A in my book, and I cannot suggest any possible changes to make this better.

Human Improvement and Selection

I chose OpenAI's improvement of Mr Garcia's, because it followed my prompt a bit more closely in not display the answers to stdout. They essentially implemented the same thing, it just so happened that OpenAI's Astra did it a little bit better. Therefore, I improved the openai.cpp version.

What I changed was pretty minimal, and I used Claude (Opus 5.5)to automate these changes on the code (because I didn't want to fix every part by hand)

This is the prompt I gave it:

```
    * Make sure every iterator is not prefix iterated
    * do not use inline if statements unless it's break or continue,
and make sure that those also are in one line, not two with a tab
and enter
    * make givensAreValid have a less vague name
    * clean up readsucess to break and err immediately if the file
is malformated
```

This is the result:

```
//////////////////////////////////////
/*
        <@
        (KU//
          "
    EECS 348 - Software Engineering
    Assignment 5
    By Lucas Scott Frias
    Description: This program, given the
    input files:
        puzzle1.txt, puzzle2.txt, puzzle3.txt
        puzzle4.txt, puzzle5.txt
    exist within the current working directory,
    will output their solutions to a file solutionN.txt
    or, if no solutions exist, output there are no solutions.
    Collaborators:
        - Claude Opus 5.5 -> used specification rewrite
        - Flash -> generated base code
        - Mr. Andy Garcia of Sofware Engineering -> wrote comments
and wrote some code

    Creation Date: 10/28/26 (the day I wrote this comment)

*/


#include <iostream> // Include input-output stream library for
console printing
#include <fstream>  // Include file stream library for reading
puzzle text files
#include <vector>   // Include vector library to store the 9x9 grid
and list of solutions
#include <string>   // Include string library for handling file
names and strings

class SudokuSolver
{                                       // Creates a class to make
a SudokuSolver
private:                                // Make it private instead
of public
    std::vector<std::vector<int>> board; // Use a 2d vector to
represent the grid of size 9x9

    bool isValid(int row, int col, int num)   // Source: Gemini with
unchanged logic; Comments by Andy Garcia
    { // Method used to check if a digit can be placed at given
index by looking through rows, columns, and 3x3 grids
        for (int i = 0; i < 9; i++)
        { // Loop through every column in the given row
```

```
            if (board[row][i] == num)
            {
                return false; // Return false if the number is
already in the row
            }
        }
        for (int i = 0; i < 9; i++)
        { // Loop through each row in the current column
            if (board[i][col] == num)
            {
                return false; // Return false if the number is
already in the col
            }
        }
        int startRow = row - row % 3; // Find the starting row index
of the 3x3 grid
        int startCol = col - col % 3; // Find the starting column
index of the 3x3 subgrid
        for (int r = 0; r < 3; r++)
        { // Loop through all 3 rows of the subgrid
            for (int c = 0; c < 3; c++)
            { // Loop through all 3 columns of the subgrid
                if (board[startRow + r][startCol + c] == num)
                {
                    return false; // Return false if it is found
                }
            }
        }
        return true; // Return true if the placement is valid across
row, col, and subgrid
    }

    // Function authored by Andy Garcia to detect if a digit is
repeated on a row or column. Error detected by Claude and given
assistance with logic and code implemented by Andy Garcia
    // Renamed from givensAreValid to givensHaveNoDuplicates to
state precisely what the check does
    bool givensHaveNoDuplicates()
    {                               // new function to test if
duplicate nums exist on the same row or col
        for (int r = 0; r < 9; r++) // loop through every row
            for (int c = 0; c < 9; c++)
            {                        // loop through every col
                int d = board[r][c]; // set d equal to the digit at
the board spot looked at
                if (d == 0) continue; // skip blanks
                board[r][c] = 0;            // take it off the board
                bool ok = isValid(r, c, d); // does it conflict with
the rest?
                board[r][c] = d;            // put it back
                if (!ok)
                {
                    return false; // two givens clash
                }
            }
        return true; // otherwise return true
    }

    int solutionCount = 0; // new private member added to count the
number of solutions done
```

```
    // Code Change #3 : edited Gemini's code to stop storing a
solution each time it was finished and instead print it off then
clear it. Storage fix discovered by Claude and code implemented by
Andy Garcia
    void solveRec(int row, int col, std::ostream &output)  // Write
each solution to the output file
    { // Recursive backtracking method. Removed solutions parameter
(Andy Garcia)
        if (row == 9)
        {                                                       //
Check if we have successfully processed past the last row
            solutionCount++;                                    //
count it
            output << "Solution " << solutionCount << ":\n"; //
Label each solution
            printBoard(output);                                  //
Write the solved board
            output << "\n";                                     //
Separate solutions
            return;                                             //
ends the function
        }
        int nextRow = (col == 8) ? row + 1 : row; // Calculate the
row index for the next step of the recursion
        int nextCol = (col == 8) ? 0 : col + 1;   // Calculate the
column index for the recusion next step

        if (board[row][col] != 0)
        {                               // Checks if cell is already
filled
            solveRec(nextRow, nextCol, output); // Go to the next
cell without changing the current
        }
        else
        { // If the current cell is blank (represented by 0)
            for (int num = 1; num <= 9; num++)
            { // Try digits 1 through 9 sequentially
                if (isValid(row, col, num))
                {                               // Check if the
digit in the loop is valid in that space
                    board[row][col] = num;      // Temporarily place
it on the board
                    solveRec(nextRow, nextCol, output); // Use
recursion to solve the next cell
                    board[row][col] = 0;        // Then backtrack
the cell to be a 0 which represent blank
                }
            }
        }
    }

public: // Public interface for the SudokuSolver class
    SudokuSolver(const std::vector<std::vector<int>> &initialBoard)
    {                       // Constructor taking initial board
        board = initialBoard; // Initialize the board by using the
provided puzzle
    }

    int findSolutions(std::ostream &output)  // Find and write all
```

```
solutions
    { // Public method to trigger solving. Updated to take no
parameters and return an int of how many solutions found (Andy
Garcia)
        if (!givensHaveNoDuplicates())
        {
            return 0; // uses new method to quit and have no
solution if a duplicate is detected (Andy Garcia)
        }
        solveRec(0, 0, output); // Start the recursion from the
first cell
        return solutionCount; // returns the solution count when
findSolutions is called
    }

    void printBoard(std::ostream &output = std::cout) const   //
Print the current board to a stream
    { // Method to print the current state of the board
        for (int r = 0; r < 9; r++)
        { // Loop through each row of the board
            for (int c = 0; c < 9; c++)
            { // Loop through each column of the row
                if (board[r][c] == 0)
                {                       // Check if the cell value is
0 (blank)
                    output << "_ "; // Print an underscore for blank
cells
                }
                else
                {                                       // If the cell
contains a valid digit
                    output << board[r][c] << " "; // Print the digit
followed by a space
                }
            }
            output << "\n"; // Print a newline at the end of each
row
        }
    }
};

class PuzzleManager
{       // Build a class to run the program and read the files
public: // These are the public methods for our file management
    void processPuzzle(const std::string &filename)  // source
Gemini; comments by Andy Garcia
    {
        std::ifstream infile(filename); // Open the file

        std::cout << "File: " << filename << "\n"; // Print the
current file name
                if (!infile.is_open())
        {
// Check if the file failed to open
            std::cout << "Error: Could not open file " << filename
<< "\n\n"; // Print out the error message
            return;
// Move to the next file
        }
```

```
        std::vector<std::vector<int>> initialBoard(9,
std::vector<int>(9, 0)); // Create 9x9 grid with all zeros
        // Code Change #4 : removed the readSuccess flag and nested
breaks, erroring and returning immediately
        // the moment a malformed character or short read is found
(Idea presented by Claude, code written by Andy Garcia)
        for (int r = 0; r < 9; r++)
        { // Go through the 9 rows of the grid
            for (int c = 0; c < 9; c++)
            {           // Go through all 9 columns in each row
                char ch; // Use a variable to hold the character
read in the file
                if (!(infile >> ch))
                {
// Try to read the file from the file stream
                    infile.close();
// Close the file stream safely
                    std::cout << "Error: Invalid puzzle in " <<
filename << "\n\n"; // will now print an error message when a file
is not correct. (Andy Garcia)
                    return;
// returns to move on to next file after detecting bad one. (Andy
Garcia)
                }
                if (ch == '_')
                {                           // Check if the
character is a blank cell
                    initialBoard[r][c] = 0; // Write a 0 to
represent blank cell on our board
                }
                else if (ch >= '1' && ch <= '9')
                {                                   // Check if
character is a digit 1 through 9
                    initialBoard[r][c] = ch - '0'; // Change that
character to an int on our board
                }
                else
                {
// Handle unexpected characters in the file
                    infile.close();
// Close the file stream safely
                    std::cout << "Error: Invalid puzzle in " <<
filename << "\n\n"; // CODE CHANGE #1 : This detects a bad character
and errors out right away (Idea presented by Claude, code written
and commented by Andy Garcia)
                    return;
// leaves the function immediately on bad input
                }
            }
        }

        infile.close(); // Close the file stream safely

        // Convert puzzleN.txt to solutionN.txt.
        std::string solutionFilename = filename;
        solutionFilename.replace(0, std::string("puzzle").size(),
"solution");
        std::ofstream solutionFile(solutionFilename);
        if (!solutionFile.is_open())
        {
```

```
            std::cout << "Error: Could not create file " <<
solutionFilename << "\n\n";
            return;
        }

        SudokuSolver solver(initialBoard);
        int count = solver.findSolutions(solutionFile);
        if (count == 0)
        {
            solutionFile << "No solution found\n";
            std::cout << "No solution found. See " <<
solutionFilename << "\n\n";
        }
        else
        {
            solutionFile << "Solutions found: " << count << "\n";
            std::cout << "Solutions found: " << count << ". See " <<
solutionFilename << "\n\n";
        }
        solutionFile.close();
    }

    void run()
    { // Main function to run through all 5 files
        for (int fileIndex = 1; fileIndex <= 5; fileIndex++)
        {
// Loops from 1 to 5 to reach all files
            std::string filename = "puzzle" +
std::to_string(fileIndex) + ".txt"; // build the name of the files
in a string
            processPuzzle(filename);
// call new function to shrink run and improve maintainability (Andy
Garcia)
        }
    }
};

int main()
{                           // Main function of the program
    PuzzleManager manager; // Creates instance of the PuzzleManager
class
    manager.run();         // Start run() to begin the process
    return 0;              // end the program by returning 0
}
```

I just mostly had it implement common sense changed (for example, the givens function now explicitly says what it does) and doing suffix incrementation, to make it easier to say.

Overall, these changes are light but I think help implement the program in a more clean way. There is not much to optimize in this case due to the work already done by Mr. Garcia and the rest of the agents to optimize this.

UML Use Case

I added the UML use case diagram in this, adding and accounting for the new addition, modifying the original use case. I used Claude Opus 5.5 to do this. This is not analysis of that Use Case, this is just

notifying you of its origin and adherence to the rubric (as is this message). You can see it in its respective PDF file and grade it thusly

Conclusion

We took a long while to analyze the initial codebase we inherented. For me, this is a guaruntee that I am doing correct analysis and chages before I make them. There are very few moments where I will be able to develop a system “tabula rasa”, so the ability to learn and understand code is what’s important, not just make these changes blindly. By reviewing a fellow classmates’ code, I gained a deeper understanding not just of this one program and prompt, but how to review and maintain code.

I think the important thing in all of this is this verifcation step. So often LLMs can generate code that seems stringed together but is actually inefficient, so I was glad to step in and implement more.

Thank you for reading this all, and I hope you have yourself a spooky and fun weekend.

Lucas Frias